# THE ENVELOPE EXPLORER

## User guide

*how to read, filter and trace every message the platform emits*

| At a glance | |
|---|---|
| **Where it runs** | http://127.0.0.1:8802 |
| **What it shows** | every envelope recorded by the lake, live |
| **Read-only** | it never writes to the platform or the bus |
| **Retains** | the most recent 20,000 envelopes in memory, plus everything on disk |
| **Built for** | forensics, compliance and model analysis |

**This guide is task-oriented. Each section answers one question: how do I find a message, how do I read one, how do I see what led to a decision, and how do I prove what happened.**

# Contents

1 Before you start

2 The screen at a glance

3 Finding the message you want

4 Reading an envelope

5 The timeline: browsing history

6 The journey: a flow diagram from a point

7 The execution trace: what a model actually used

8 How and why a model traded — or did not

9 Roles and redaction

10 Three worked examples

11 Keyboard and quick reference

12 Troubleshooting

# 1 Before you start

The Explorer is a viewer. It reads the lake that the recorder writes; it does not talk to the exchange and it cannot change anything. If it is empty, the problem is upstream, not here.

## 1.1 What should be running

| Service | Port | Why the Explorer needs it |
|---|---|---|
| **market data** | :8796 | the source of every price, depth and candle envelope |
| **recorder** | :8801 | consumes the bus once and writes the lake the Explorer reads |
| **envelopeapi** | :8802 | the Explorer itself |

Start them in that order. The Explorer will start without the recorder, but it will have nothing to show.

## 1.2 Opening it

1. Start the core, then the Explorer (scripts/start.sh does both in order).
2. Open http://127.0.0.1:8802 in a browser.
3. The header should show a rising envelope count within a few seconds.

**If the header shows 20,000 and stops climbing, the Explorer is fine and the feed has stopped. Check the recorder's /stats before touching the Explorer.**

# 2 The screen at a glance

Four regions, always in the same place.




*Figure 1 — the Explorer with an envelope selected: filters on the left, the list in the middle, the envelope's anatomy on the right.*

| Region | What it is for |
|---|---|
| **Header** | counts: envelopes held, bytes, planes, topics. The 'viewing as' role is on the right. |
| **Timeline** | a histogram of traffic over the retained window. Drag across it to select a time range. |
| **Left panel** | filters: plane, topic, symbol, producer, correlation id, free text. |
| **Middle** | the list of envelopes (newest first) or the journey flow diagram. |
| **Right** | the anatomy of the selected envelope: meta, payload, references, chain. |

## 2.1 Two tabs in the middle

ENVELOPES is a list, one row per message. JOURNEY is a flow diagram of how traffic moves through the platform, from market data to advisories to decisions. Escape always takes you back to the previous view.

# 3 Finding the message you want

Filters combine: each one narrows the list further. The count above the list tells you how many matched.




*Figure 2 — the plane filter set to intel. Every row shows a green 'INTEL' pill and the topic counts on the left change with it.*

| Filter | Use it when | Example |
|---|---|---|
| **Plane** | you want one class of traffic only | market, intel, model, control, system |
| **Topic** | you know the exact kind of message | intel.forecast, model.decision |
| **Symbol / key** | you are following one instrument | BTCUSDT |
| **Producer** | you suspect one component | market_data, signals, tad |
| **Correlation id** | you have an id from a log or a report | cid-9f2a41… |
| **Contains** | you remember a value but not the topic | 0.0431 or 'liquidation' |

Filters match case-insensitively and apply to the symbol, producer, correlation id and free text. The topic list under 'by volume' is also a filter: click a topic to select it.

## 3.1 When nothing matches

An empty list is information. The header count drops to zero, the list says so plainly, and the timeline above it keeps filling — which tells you the feed is alive and the filters are the reason you see nothing.

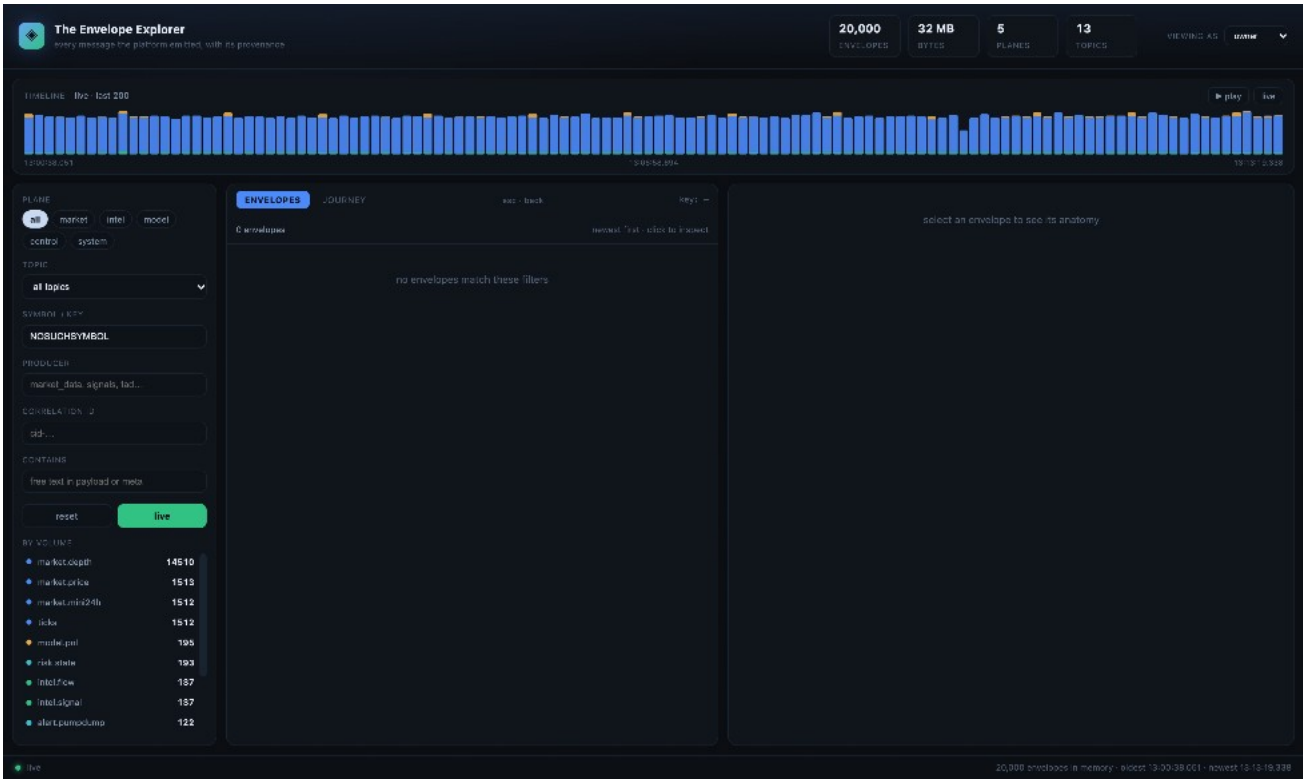



*Figure 8 — a symbol filter that matches nothing: 0 envelopes, while the timeline keeps receiving traffic.*

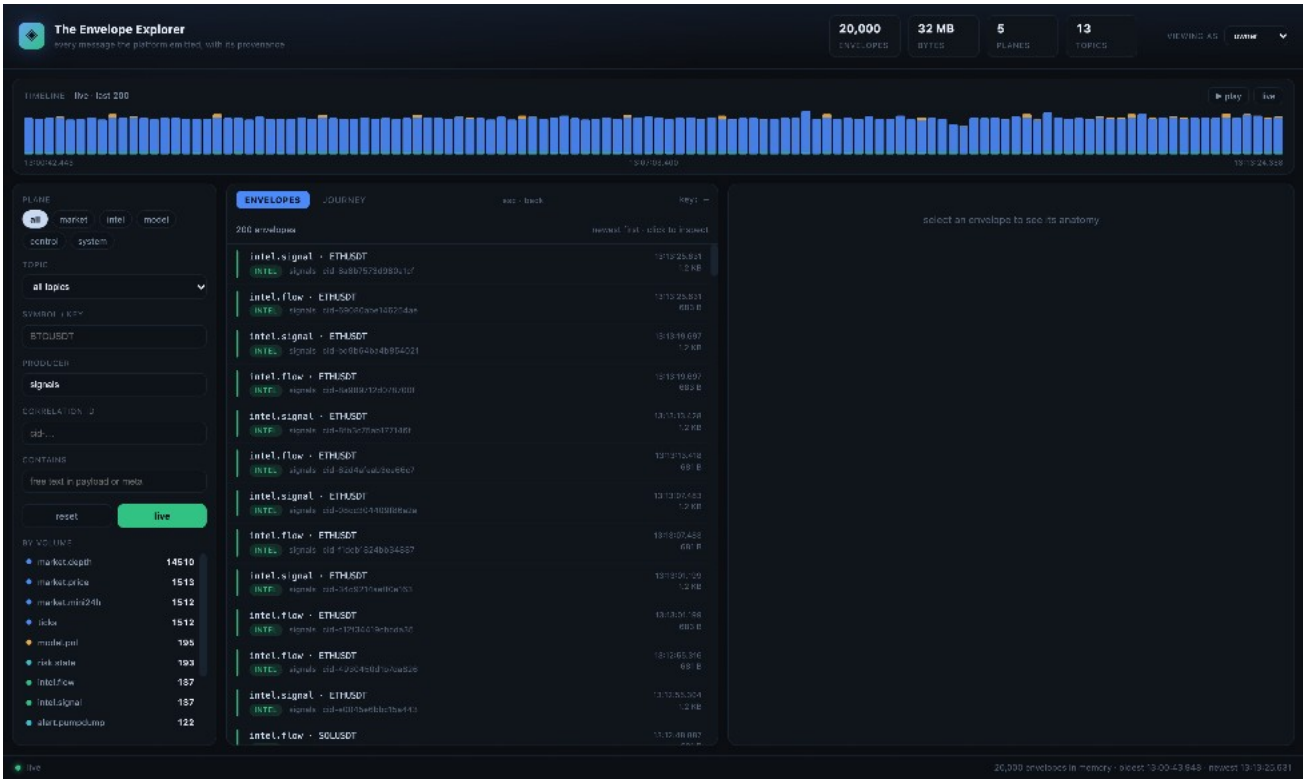



*Figure 9 — filtering by producer: every row is a message that component wrote.*

## 3.2 Live versus paused

The green LIVE button means new envelopes are being prepended as they arrive. Set to PAUSED to freeze the list while you read. Selecting a time range on the timeline also pauses it, because you are no longer looking at 'now'.

# 4 Reading an envelope

An envelope has three members and the right-hand pane shows all three, always: data, meta and references. A strip at the top of the pane tells you at a glance which are present.

## 4.1 The three members

| Member | What it carries | If it is missing |
|---|---|---|
| **data** | the payload for this topic | never — a message with no data is refused |
| **meta** | version, plane, producer, generation, correlation id, computed_at, recorded_at, bytes | the pane says 'inferred' |
| **refs** | topic, key, generation and correlation id of each input | the pane says 'none' and explains what a reference would contain |

References come in two kinds, and the pane always says which: **declared** by the producer, or **derived** by the recorder. The books do not declare their inputs yet, so the recorder attaches, for every model decision, the newest envelope of each input topic for that symbol in the preceding five minutes — and marks each one derived, with its age. A reference is never silently presented as declared when it is inferred.

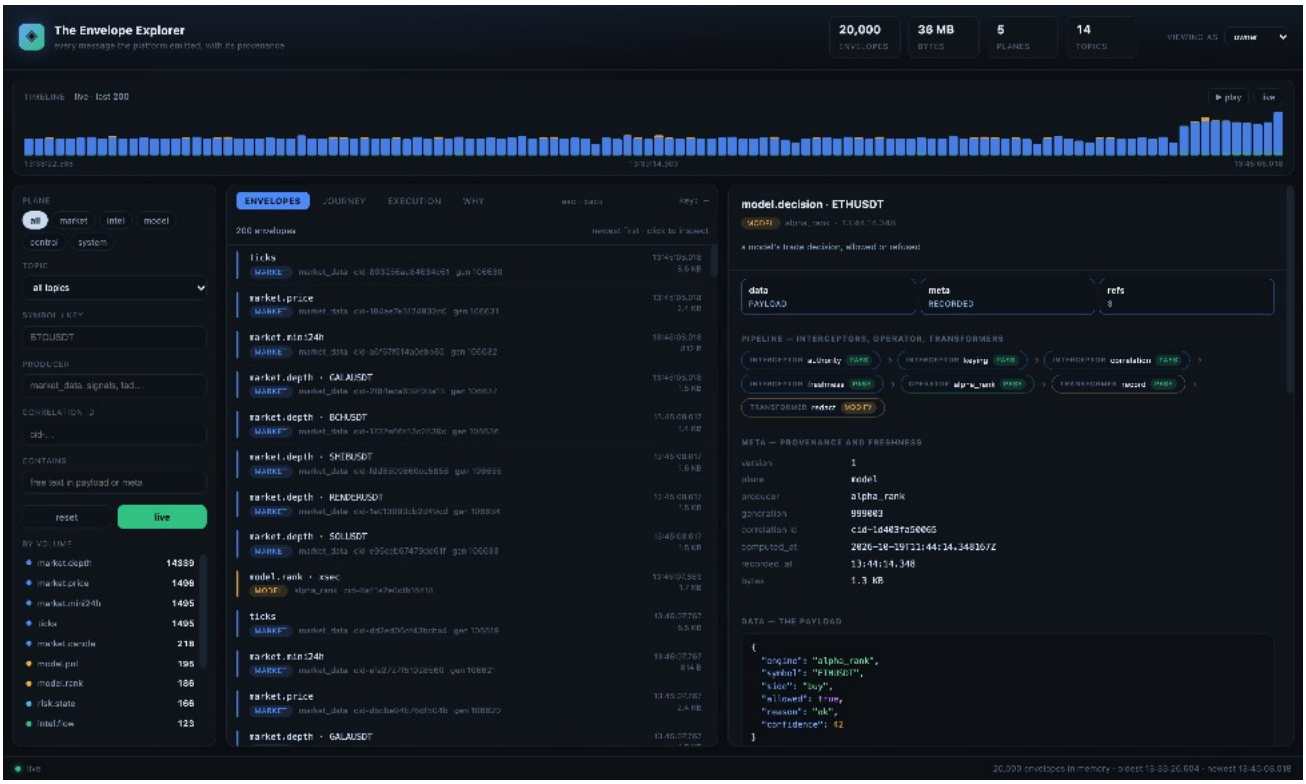



*Figure 6 — a decision with three derived references: market.depth at 0.2 s, intel.signal and intel.flow at 1.8 s. The refs member shows 3, and the section names its own provenance.*

When a producer publishes its own wrapper (the Python books do: topic, producer, data, version, seq), the pane unwraps it: the Data section shows the inner payload and names the topic and producer it was published under.

## 4.2 Meta — provenance and freshness

| Field | Meaning | Why it matters |
|---|---|---|
| **version** | the envelope contract revision | a reader refuses a revision it does not understand |
| **plane** | market, intel, model, control or system | decides priority and how it is handled |
| **producer** | the component that wrote it | the first question in any incident |
| **generation** | the producer's own counter for that topic | gaps reveal a dropped producer |

| | | |
|---|---|---|
| **correlation id** | the identity that follows the message | resolves the chain around it |
| **computed_at** | when the payload was produced | this is what staleness gates refuse on |
| **recorded_at** | when the recorder stored it | the difference is transport and queue time |
| **bytes** | size on disk | market data is bytes, and bytes have a budget |

## 4.3 Data — the payload

The payload exactly as it travelled, syntax-coloured. It is never rewritten by the Explorer: what you see is what the recorder stored.




*Figure 6 — one decision envelope: meta on top, payload in the middle, chain underneath.*

## 4.4 Pipeline — interceptors, operator, transformer

Every envelope carries the stages it passed through: the interceptors that checked it, the operator that produced it, and the transformers that changed its shape. This is the ATLAS pipeline made visible on the packet itself rather than in a log beside it. A refusal is recorded as an envelope too, so 'why not' is a packet you can open rather than a line you have to find.

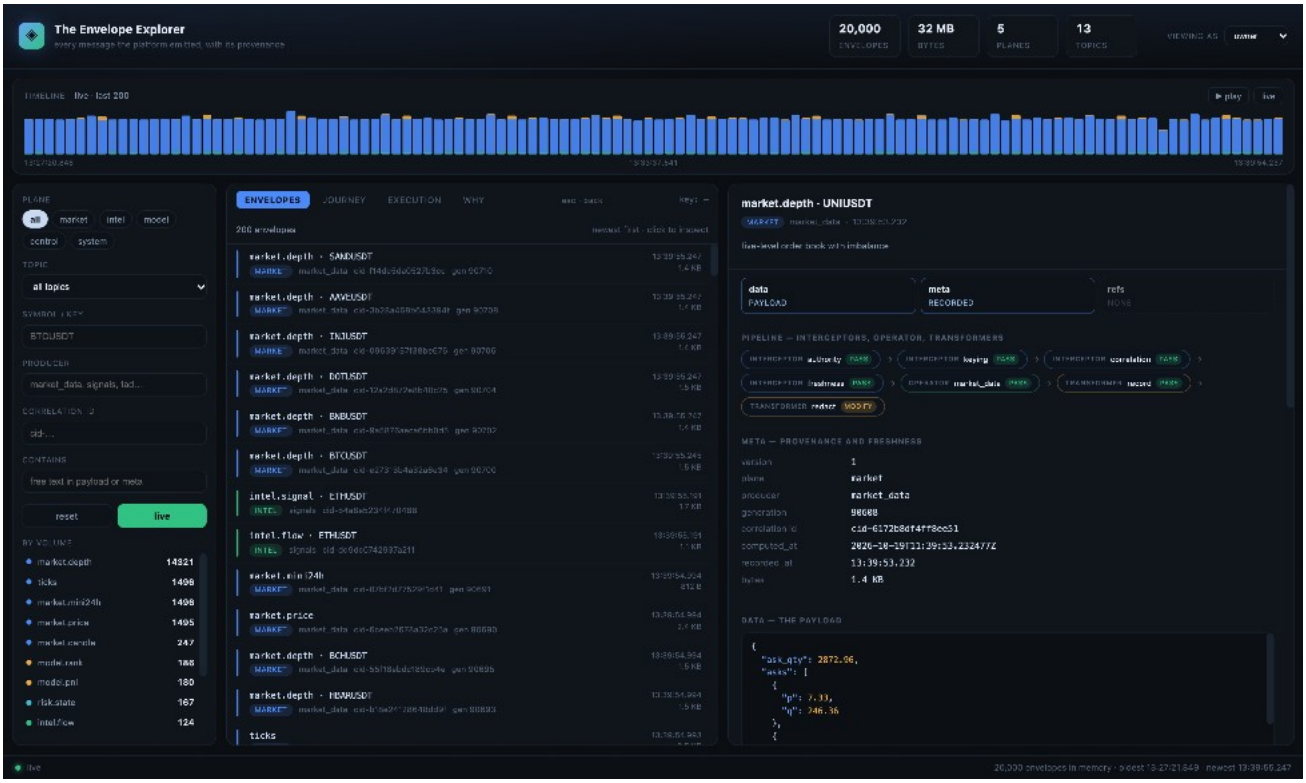



*Figure 7 — the pipeline strip: four interceptors, the operator, and two transformers, each with its outcome. The last one is the view-time redaction applied for this role.*

| Stage | What ran | What the chip means |
|---|---|---|
| **interceptor** | authority, keying, correlation, freshness | a pass means the rule was satisfied; a refusal names the rule and the number |
| **operator** | the producer (market_data, signals, a model) | the capability that produced this payload |
| **transformer** | record, redact | a change of shape: stored once, or redacted for the viewing role |

Outcome colours: green pass, red refuse, amber modify. Hover any chip for its detail — for a refusal that is the exact text the governor recorded, for a transformer it is what changed.

## 4.5 Chain — what else happened for that symbol

Every envelope recorded for the same symbol, oldest first, with arrows. Click any step to jump to it. This is the quick way to walk a symbol's recent history without leaving the page.

**A message with no key (for example market.mini24h) has no chain. That is not a bug: it is a multi-symbol snapshot, so there is no single chain to show.**

# 5 The timeline: browsing history

The histogram above the list is traffic over time, one bar per slice, coloured by plane. It is how you find a moment rather than a message.




*Figure 3 — a time range selected by dragging across the bars. Everything outside dims, the header names the window, and the list narrows to it.*

1. Drag across the bars to select a window. The list reloads to that window and the live pill turns to paused.
2. Press ▶ play to step the window forward, so the retained period plays back like a recording.
3. Press 'live' to return to the newest 200 envelopes.

Two details worth knowing. The selection survives the histogram's own refresh, so the bars do not jump under the cursor while you are reading. And a gap in the histogram is a real gap: the Explorer never interpolates traffic that was not recorded.

# 6 The journey: a flow diagram from a point

This is the view to use when the question is 'how did this happen', rather than 'what is this message'.

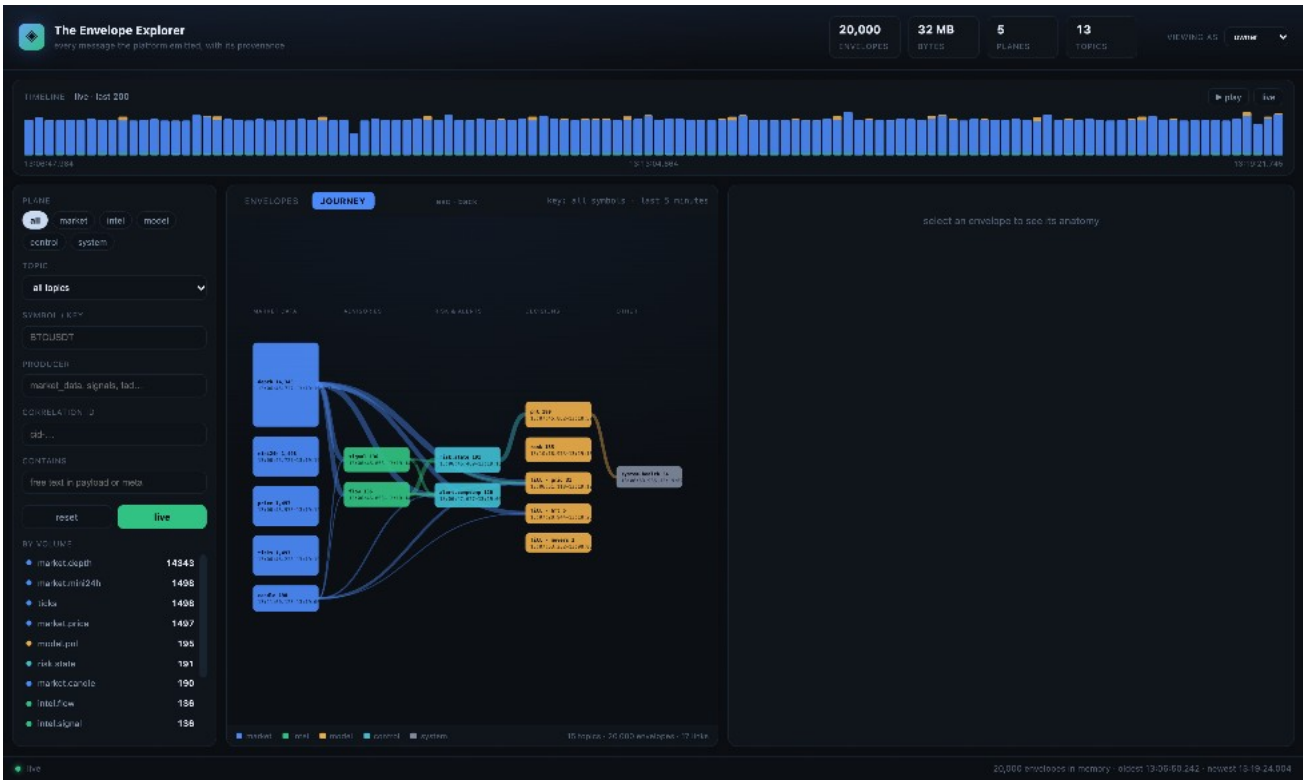



*Figure 4 — the whole platform as a flow: five stages and fifteen nodes, including each model that wrote a fill. Box height is volume; ribbon width is the number of correlations observed.*

## 6.1 Two ways to open it

| Way | What you get |
|---|---|
| **Select an envelope, then open JOURNEY** | the flow opens at that message: two minutes of lead-in, five minutes forward, on its symbol. The node containing it is ringed. |
| **Open JOURNEY with nothing selected** | the last five minutes across all symbols — a live view of the platform's shape. |

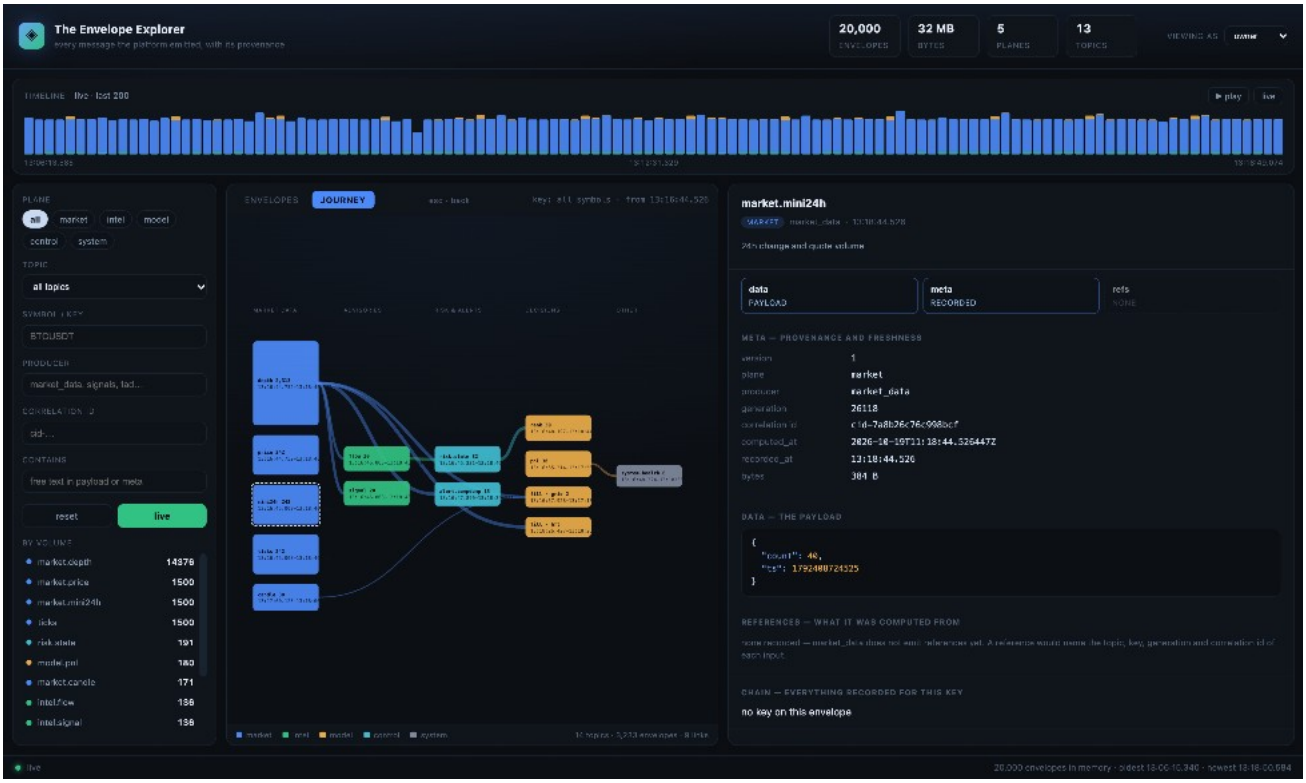



*Figure 5 — the same diagram anchored on a model.fill envelope: the dashed ring marks where you came in, and the window runs forward from that moment.*

## 6.2 Reading the diagram

- Columns are the pipeline, left to right: market data → advisories → risk & alerts → decisions → other.
- Box height is volume. market.depth is tall; a handful of fills is a sliver.
- Box label is the topic and its count; the small line beneath is the time span those messages cover.
- Ribbon width is the number of observed correlations between the two topics.
- Colour is the plane: blue market, green intel, amber model, teal control, grey system.
- Nodes are split by producer, so three models writing model.fill appear as three boxes, each labelled with its engine.
- The dashed ring marks the anchor — the message you came in on.

## 6.3 Moving around

| Action | What happens |
|---|---|
| **Click a box** | opens the newest envelope of that topic in the right-hand pane |
| **Double-click a box** | re-anchors the flow on that node and walks forward from it |
| **Select another envelope in the list** | re-anchors the flow there |
| **Esc** | back: returns to the previous view and the state you were in — selection, scroll and filters included |
| **Brush the timeline** | the flow uses that window too |

## 6.4 A worked walk

1. Find a model.decision row in the list (filter plane=model, topic=model.decision).
2. Open JOURNEY. The ring shows where the decision sits in the flow.
3. Read left: the market data and advisories that preceded it.
4. Double-click the fill box to re-anchor on the fill and see what followed it.

**Honest caveat: ribbons are temporal correlations — 'next stage for the same symbol within 120 seconds' — not declared references, because the Python books do not emit refs yet. When they do, the same diagram becomes literal lineage with no**

interpretation required.

# 7 The execution trace: what a model actually used

The journey answers 'how does traffic flow'. The execution trace answers a narrower and more useful question: for this one fill, which raw envelopes did the model read, in what order, and how old were they when it acted?

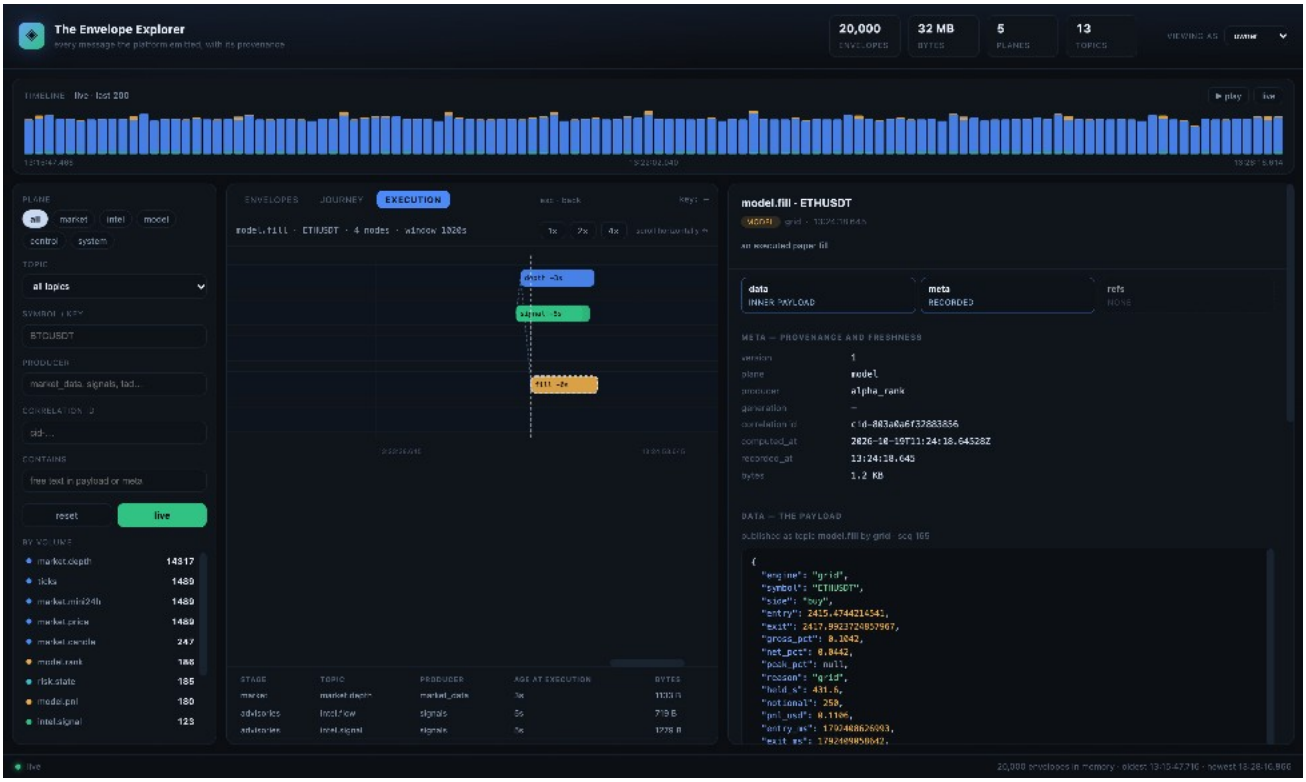



*Figure 10 — the execution trace for one model.fill: lanes by stage, nodes placed in time, the dashed line marking the moment of execution, and the inputs listed beneath with the age each had when the model acted.*

## 7.1 How to read it

| Element | Meaning |
|---|---|
| **Lanes** | one row per stage: market, advisories, risk, model, other |
| **Node** | one envelope. The label is the topic and how old it was at execution (−1s means one second before the fill) |
| **Dashed vertical line** | the moment of execution — the anchor envelope |
| **Dotted path** | the reading order: stage by stage through the chosen envelopes |
| **1× / 2× / 4×** | zoom. The canvas scrolls horizontally, so a fifteen-minute window stays readable |
| **Table beneath** | the inputs the anchor used: stage, topic, producer, age at execution, bytes |

## 7.2 Working through an execution

1. Open the EXECUTION tab with a fill selected (filter topic = model.fill to find one).
2. The view scrolls itself to the moment of execution and marks it with a dashed line.
3. Read left to right: market data first, then the advisories, then anything that followed the fill.
4. Click any node to see its raw envelope in the right-hand pane — data, meta and references.
5. Use the table beneath to compare ages: a decision made on a 600-second-old candle is a different decision from one made on a 2-second-old candle.

**This is the view for 'why did the model do that'. It shows the exact envelopes, their ages, and their producers — not a summary of them. When books start emitting references, the dotted path becomes literal lineage instead of a time-ordered reading of the**

window.

# 8 How and why a model traded — or did not

This is the question a reviewer actually asks. The WHY tab answers it directly: every decision the models recorded, split into the trades they took and the trades they refused, with the rule that stopped them.

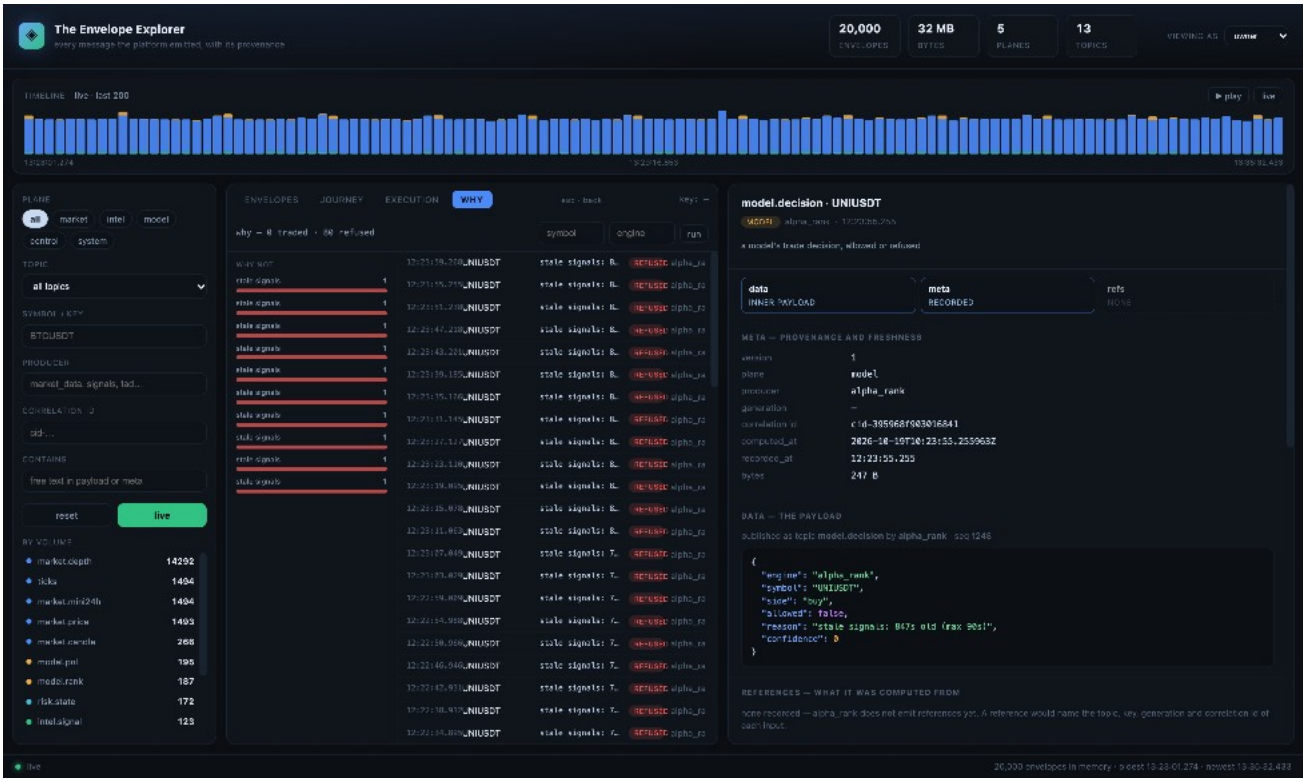



*Figure 11 — the WHY tab: reason tally on the left, decisions on the right, and the raw refusal envelope open beside them.*

## 8.1 Reading the tab

| Part | What it tells you |
|---|---|
| **Header** | how many decisions the scan found: traded, refused, and the symbol if filtered |
| **Why not (left)** | the refusal reasons, tallest first. 'stale signals' dominating means the feed is late, not that the model is picky |
| **Decision rows** | time, symbol, reason as the model recorded it, outcome, engine |
| **Right pane** | the raw decision envelope: data, meta, references — including allowed and reason verbatim |
| **Filters** | symbol and engine; Enter runs the scan |

## 8.2 What a decision actually contains

A model.decision payload is short on purpose. It carries the engine, the symbol and side, whether the governor allowed it, the reason, and the confidence the model had. Everything else is in the envelope: who produced it, when, and the correlation id that ties it to the rest of the chain.

| Field | Example | Why it matters |
|---|---|---|
| **engine** | alpha_rank | which model made the call |
| **symbol / side** | UNIUSDT · buy | what it wanted to do |
| **allowed** | false | the governor's verdict |
| **reason** | stale signals: 847s old (max 90s) | the rule and the number that tripped it |
| **confidence** | 0 | the model's own conviction — 0 means none |

## 8.3 From refusal to evidence in three clicks

1. Open WHY and find the refusal you care about — the reason is already visible in the row.
2. Click it: the right pane shows the raw envelope, so the reason is quoted from the record, not paraphrased.
3. Double-click it: the EXECUTION tab opens on that decision, with the inputs it read and how old each was.

**A refusal is evidence, not noise. 'stale signals: 847s old' names the rule, the age and the limit, and the envelope that says it is the one the platform stored. That is the difference between 'it did not trade' and 'we cannot explain why it did not trade'.**

# 9 Roles and redaction

The 'viewing as' selector in the header changes what you are allowed to see. It is a real filter applied on the server, not a cosmetic setting.

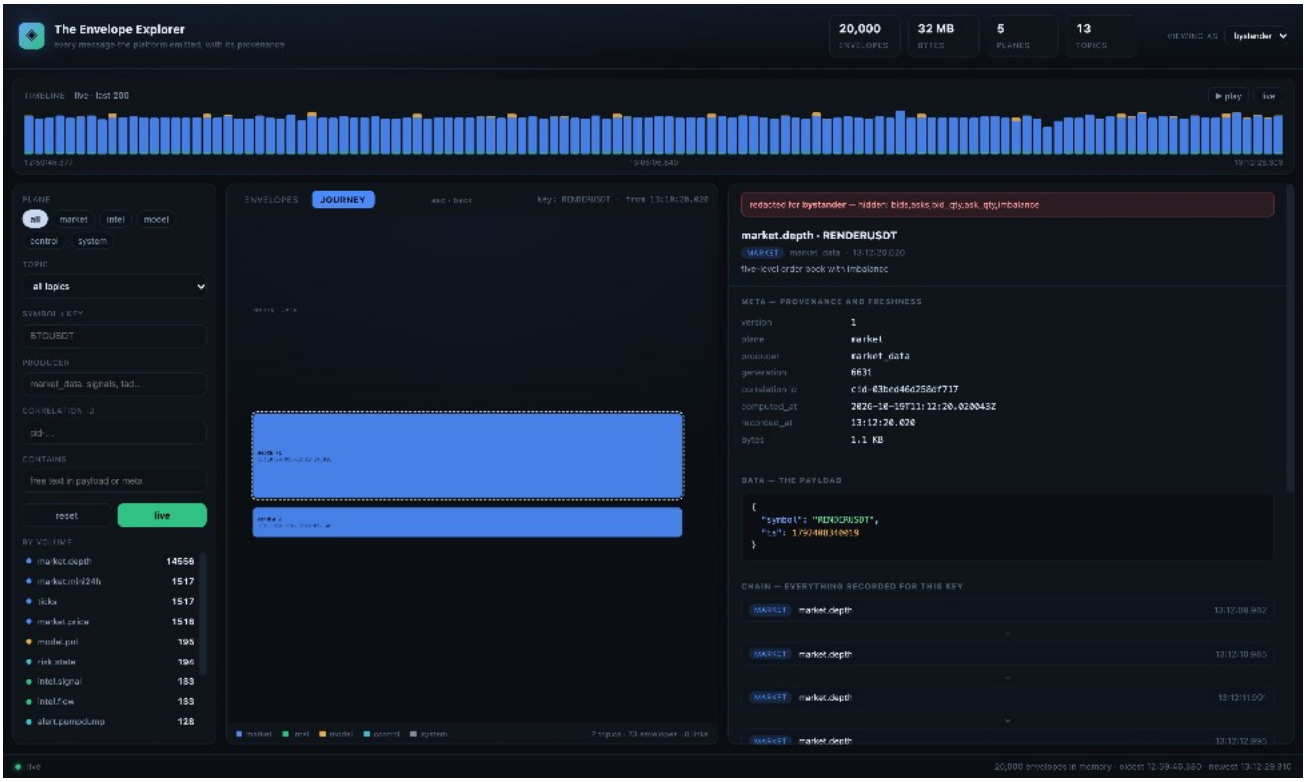



*Figure 12 — the same envelope viewed as a bystander: depth fields removed, and a red band naming exactly what was withheld.*

| Role | Sees | Typical use |
|---|---|---|
| **owner** | everything | you are the operator |
| **auditor** | prices and sizes removed, everything else | reviewing decisions without market detail |
| **bystander** | the shape and provenance only | showing status to someone outside the desk |

Every redaction is recorded in the envelope's meta as redacted_for and redacted_fields, so the removal is part of the record rather than a silent edit. If someone asks what you were shown, the answer is in the message.

# 10 Three worked examples

## 10.1 Why did we buy this?

1. Filter symbol = BTCUSDT and plane = model.
2. Open the model.decision row you care about.
3. Switch to JOURNEY: the ring sits on the decision; read left for the advisories and market data.
4. Note the correlation id from the meta pane — quote it in any discussion.

## 10.2 What happened at a specific minute?

1. Drag the timeline across that minute.
2. The list narrows to it; sort your reading by plane using the left panel.
3. Watch for a gap rather than a quiet period: a gap means no producer, not no market.

### 10.3 Which model is producing decisions?

1. Filter producer = books (or a specific engine) and topic = model.decision.
2. Compare the generation numbers: a stall shows as a flat generation while others climb.
3. Cross-check the recorder's per-topic counts in /stats to confirm delivery, not just display.

# 11 Keyboard and quick reference

| Key or control | Effect |
|---|---|
| **Esc** | back: restores the previous view *and* the envelope that was selected; pressed with no history it clears the selection |
| **Tab (browser)** | focus the next filter |
| **live / paused** | follow new envelopes, or freeze the list |
| **▶ play** | step the selected window forward |
| **reset** | clear every filter and return to live |

## 11.1 What the colours mean

| Colour | Plane | Typical topics |
|---|---|---|
| **blue** | market | ticks, market.price, market.depth, market.candle |
| **green** | intel | intel.signal, intel.flow, intel.ta, intel.forecast |
| **amber** | model | model.decision, model.fill, model.pnl, model.rank |
| **teal** | control | risk.state, exec.quote, alert.pumpdump |
| **grey** | system | system.health, review.verdict |

# 12 Troubleshooting

| Symptom | Likely cause | What to do |
|---|---|---|
| **List is empty** | nothing matches the filters | press reset |
| **Count frozen at 20,000** | feed or recorder stopped | check :8801/stats and :8796/health |
| **A topic never appears** | its producer is silent | intel.forecast needs 40 bars before it publishes |
| **Journey shows one column** | the symbol only has market data | pick a symbol with decisions (filter plane=model first) |
| **Ribbons are missing** | no correlations in the window | widen the window or pick a busier symbol |
| **Chain says 'no key'** | the topic is a multi-symbol snapshot | expected for market.mini24h and ticks |

**The Explorer is read-only by construction: it opens the lake, never the bus, and never writes. If something looks wrong in the platform, the Explorer is the wrong place to fix it — but the right place to prove it.**